Bin Packing de Reservas de Frete com Custo Variável em uma Rota Logística: Uma Primeira Solução Exata

TODO: Substituir Pelo Autor

***Resumo*—Instanciamos um modelo de empacotamento de caixas com custo variável (VSBPP) sobre dados reais de frete: o dataset "Supply Chain Logistics Problem" da Brunel (pedidos, brackets tarifários e tarifas multi-transportadora, hospedado no Kaggle). Para uma rota de envio, pedidos são itens, reservas (*bookings*) são caixas, e classes tarifárias (transportadora $\times$ serviço $\times$ bracket de peso) são classes de caixa com capacidade C_k, carga mínima faturada l_k, taxa fixa f_k e tarifa por kg R_k — o custo de um embarque é $\max(f_k,\ R_k W)$, com o `minimum cost` atuando como piso. Declaramos os domínios de dados, as variáveis, o objetivo e as restrições, resolvemos uma instância pequena de forma exata e comparamos contra uma heurística first-fit-decreasing e um limite inferior baseado em peso.**

***Index Terms*—bin packing, empacotamento com caixas de tamanho variável, programação inteira mista, logística, consolidação de frete**

I. Introdução

Bin packing empacota itens de tamanhos w_i em caixas de capacidade C minimizando o número de caixas, e é NP-difícil [1] [2]. Na **variante de caixas de tamanho (e custo) variável**, as caixas vêm em classes com capacidades e preços distintos, e o objetivo é o custo total das caixas usadas [3] [4]. Neste projeto as classes não são sintéticas: elas são *lidas de uma tabela de tarifas de frete real*, cada faixa transportadora $\times$ serviço $\times$ bracket de peso do Supply Chain Logistics Dataset [5].

Para uma rota de envio (o, d), uma **reserva** (*booking*; um envio de frete contratado junto a um serviço de transporte; o dataset não tem conceito de caminhões físicos, e o modo predominante é aéreo) desempenha o papel de caixa. Seu peso faturável deve cair dentro de um bracket tarifário: o teto do bracket é a capacidade da reserva e a sua tabela de preços cobra $\max(R_k\, w,\ f_k)$ — a tarifa por kg `rate` sobre o peso **total** do embarque, com piso no `minimum cost` do bracket. A otimização decide tanto o empacotamento quanto a classe de cada reserva; a atribuição de transportadora por pedido do dataset é uma baseline de entrada, não uma restrição (e uma transportadora, `V44_3`, fica sem tarifa nos dados e precisaria ser reatribuída de qualquer forma).

II. Dados e construção de instâncias

A. Fonte e esquema

Sete tabelas relacionais [5]; as relevantes aqui: `OrderList` (9{,}215 pedidos; usamos `Origin Port`, `Destination Port`, `Carrier`, `Service Level`, `Unit quantity`, `Weight`) e `FreightRates` (1{,}540 linhas tarifárias; usamos carrier, `orig_port_cd`, `dest_port_cd`, `svc_cd`, `minm_wgh_qty`, `max_wgh_qty`, `minimum cost`, `rate`).

TODO: Substituir Pelo Autor is with Instituto de Ciências Matemáticas e de Computação (ICMC), Universidade de São Pauloe-mail: author@usp.br

B. Pré-processamento

1. **Normalização de unidades.** Os pesos dos pedidos são lidos em quilogramas. Os brackets de algumas transportadoras são cotados em libras (limites incrementados de exatamente 1 lb = 0.453592); seus limites são convertidos para kg.
2. **Construção de classes.** Por rota, as linhas de `FreightRates` geram classes $k = (\text{carrier}, \text{service}, \text{bracket})$ com \$C_k = \$ `max_wgh_qty` (kg), \$l_k = \$ `minm_wgh_qty` (kg), \$f_k = \$ `minimum cost` e \$R_k = \$ `rate` (US\$/kg).
3. **Filtro de dominação de classes.** A classe k é descartada quando outra classe k' tem $C_{k'} \geq C_k$, $f_{k'} \leq f_k$ e $l_{k'} \leq l_k$ — ela nunca pode ser estritamente útil.
4. **Pedidos de peso zero** (2 de 9{,}215) são *mantidos*: podem viajar em qualquer reserva aberta a custo marginal zero e são pós-atribuídos após a resolução.

C. Alegações de domínio usadas abaixo

- I: pedidos da rota escolhida (pesos dos itens $w_i \geq 0$, kg).
- K: classes tarifadas da rota (definição acima, após filtragem).
- $J = \{1, \ldots, n\}$: slots candidatos de reserva — nenhuma rota precisa de mais reservas do que pedidos.

III. Modelagem Matemática

Em nosso trabalho, adaptamos o problema proposto pelo artigo de referência — a otimização do roteamento de veículos (*Vehicle Routing Problem*, VRP) — impondo um limite ao número de paradas por viagem. No problema original, admite-se que um veículo realize várias entregas ao longo do trajeto até o destino, o que permite reduzir ao mesmo tempo o número de veículos e a distância total percorrida [6], [7]. O carregamento dos pedidos é considerado, mas o custo por distância percorrida é o fator predominante na precificação das viagens. Ao limitar o número de paradas a **uma**, todas as viagens tornam-se diretas e o problema de roteamento degenera em um **problema de**

empacotamento com caixas de tamanho variável (*Variable Sized Bin Packing Problem*, VSBPP) [3], [8], [9]. Noutras palavras, ao resolvermos o VSBPP, estipulamos o teto dos custos que o VRP pode reduzir.

O VSBPP difere do problema clássico de empacotamento em caixas por admitir compartimentos de capacidade variável. No caso do nosso dataset, os compartimentos são caminhões com distintas restrições de área e carga. No vocabulário do domínio: itens são **pacotes**; caixas são **caminhões** de quatro tipos (identificados por sua extensão); e a **instância** é uma rota $(\text{CD} \to d)$ em viagem direta para 61 destinos distintos.

Como cada pacote consome dois recursos do caminhão — área e carga —, a instância é **multidimensional** [10], [11]. Isso a torna mais difícil que o caso unidimensional. Um **esquema de aproximação polinomial assintótico** (APTAS) é uma família de algoritmos que, para todo $\varepsilon > 0$, garante solução a no máximo $(1+\varepsilon)$ vezes o ótimo mais uma constante, em tempo polinomial no número de itens. Para o empacotamento vetorial em duas ou mais dimensões tal esquema **não existe**, a menos que P = NP [12]; em uma dimensão ele existe tanto para o BPP clássico [13], [14] quanto para o VSBPP [8], [9].

Ser NP-difícil, porém, não torna o MILP inútil: BPP e VSBPP já são NP-difíceis [1] e solvers exatos resolvem instâncias grandes quando a formulação é boa. O que pesa aqui é a combinação de duas dimensões com a escolha da classe: não há garantia de aproximação e o esforço exato cresce rápido com o número de itens. Por isso usamos a **programação linear inteira mista** (MILP) exata nas instâncias pequenas e limites inferiores com heurísticas nas maiores.

Esta seção formaliza um **VSBPP multidimensional** com frota heterogênea, instanciado no dataset de roteirização da Microsoft [6], [7]. A formulação segue os termos padrão de *bin packing* [3] [15] e acrescenta duas generalizações: as caixas vêm em **classes** com capacidade em **duas dimensões** e custo fixo por viagem; e cada item tem uma **janela de tempo** (disponibilidade e prazo).

A. Conjuntos

- $I = \{1, \ldots, n\}$: pacotes destinados a uma dada rota
- $T = \{1, \ldots, 4\}$: tipos de caminhão disponíveis
- $J = \{1, \ldots, m\}$: *slots* (posições) de caminhão disponíveis, com $m \leq n$ (um caminhão aberto leva ao menos um pacote)

Um **slot** não é um caminhão específico, é uma **posição abstrata** que o modelo pode deixar vazia ou preencher com um caminhão de algum tipo. Como o **número** de veículos é justamente o que se quer decidir, criam-se m posições candidatas e deixa-se que y_{jt} indique se a posição j é um caminhão do tipo t; se nenhum tipo for atribuído, o slot permanece fechado (R4–R5). Os slots são intercambiáveis entre si — trocar dois produz a mesma solução física —, o que gera **simetria de permutação**: cada plano de carregamento tem até $m!$ representações de custo idêntico e de mesmo limitante [16]. A simetria não altera o valor ótimo, mas encarece a prova de otimalidade. Como cópias simétricas têm o mesmo limitante, o *branch-and-bound* não consegue podar uma a partir do resultado da outra, e o esforço passa a acompanhar o número de rótulos em vez do número de soluções distintas [17]. A relaxação linear também fica degenerada, com vértices fracionários equivalentes [18], e as desigualdades válidas obtidas para uma cópia não se transpõem automaticamente às demais [16] — o que motivou as técnicas de quebra baseadas em órbitas [17], [19]. Por isso o modelo inclui as restrições de quebra R9–R11: R9–R10 tratam da permutação de slots e R11 da repetição de pacotes idênticos.

B. Parâmetros

O *dataset* de referência nos fornece 20 colunas distintas, das quais **11** são incorporadas enquanto parâmetros em nossa modelagem. Esta correlação está descrita nas quatro tabelas a seguir, sobre devemos fazer algumas considerações importantes.

a) Área enquanto uma grandeza vetorial:

Para cada pacote, o dataset fornece `Area` (m²) e `Weight` (kg), não comprimento, largura ou altura. A capacidade é **vetorial** (soma de recursos), não um empacotamento geométrico; empilhamento não é representado.

b) Coluna `Danger_Type`:

A coluna `Danger_Type` foi removida da modelagem por vacuidade. Esta coluna descreve tipos de pacotes (`type_1` e `type_2`) os quais, por considerações de segurança, não podem ser transportados juntos. Entretanto estes pacotes não são solicitados a um mesmo destino sequer uma vez, então esse encontro jamais poderia ocorrer na atual configuração.

c) Grandezas de tempo:

As grandezas — av_i, dl_i, τ_j, Δ e M — são medidas em **horas a partir de uma origem comum** (o menor `Available_Time` da instância): o dataset traz carimbos de data e hora, que precisam virar números para entrar nas restrições lineares, e a hora é a unidade coerente com v (km/h).

d) Classes:

A **classe** é o tipo de caminhão t, com (A_t, W_t, C_t). O custo por unidade de capacidade é monotônico no tamanho (0,332 por m² no caminhão de 7,6 m contra 0,248 no de 16,5 m), mas o custo **por caminhão** cresce com o porte — é esse *trade-off* que dá conteúdo ao VSBPP: para cargas pequenas, um caminhão menor é mais barato apesar de menos eficiente por unidade.

e) Janela de consolidação:

Δ **não é um dado**: é um parâmetro de política herdado do acelerador de referência. No `README` do repositório [6], ela aparece como a terceira restrição do problema:

> "A diferença de tempo disponível entre os itens disponíveis mais cedo e mais tarde para um mesmo caminhão deve ser definida pelo usuário (exemplo: 4 horas)" [Tradução nossa]

Adotamos o valor sugerido, $\Delta = 4$ h. As restrições 2 e 4 do mesmo `README` fundamentam, respectivamente, R6 (o caminhão só parte quando todos os itens estão disponíveis) e R8 (entrega antes do prazo). Não fosse especificado, um caminhão poderia esperar dias para consolidar carga. No mais, o dataset não precifica espera; por isso o modelo espera até o limite dos prazos (R7–R8), e essa ausência é apenas registrada.

Parâmetro	Descrição	Coluna	Unidade
i	índice do pacote	`Item_ID`	—
a_i	área ocupada pelo pacote i (≥ 0)	`Area`	m^2
w_i	peso do pacote i (> 0)	`Weight`	kg
av_i	instante de disponibilidade do pacote i (≥ 0)	`Available_Time`	h
dl_i	prazo de entrega do pacote i (≥ 0)	`Deadline`	h
—	origem da rota (constante `City_61`)	`Source`	—
—	destino da rota (define a instância)	`Destination`	—
—	identificador de pedido; não entra no modelo	`Order_ID`	—
—	identificador de material; não entra no modelo	`Material_ID`	—
—	classe de perigo; excluída por vacuidade	`Danger_Type`	—

Tabela I: Colunas de `order_large.csv` e `order_small.csv`: 10 colunas, 5 delas viram parâmetros.

Parâmetro	Descrição	Coluna	Unidade
—	origem (confirma `City_61`)	`Source`	—
—	destino (confirma a instância)	`Destination`	—
d	distância da rota (> 0)	`Distance(M)`	km

Tabela II: Colunas de `distance.csv`: apenas `Distance(M)` vira parâmetro.

Parâmetro	Descrição	Coluna	Unidade
t	índice do tipo de caminhão	`truck_type`	—
A_t	capacidade de área do caminhão do tipo t (> 0)	`area_capacity_m2`	m^2
W_t	capacidade de peso do caminhão do tipo t (> 0)	`weight_capacity_kg`	kg
c_t	custo por quilômetro do tipo t (> 0)	`cost_per_km`	US$/km
v	velocidade do caminhão	`speed_kmh`	km/h
—	comprimento (usado apenas para derivar/validar A_t)	`inner_length_m`	—
—	largura (usado apenas para derivar/validar A_t)	`inner_width_m`	—

Tabela III: Colunas de `trucks.csv`: cinco parâmetros definem a frota.

Parâmetro	Descrição	Unidade
C_t	custo fixo de um caminhão do tipo t ($= c_t\, d$)	US$
t_{viagem}	tempo de viagem na rota ($= d/v$)	h
Δ	janela de consolidação ($= 4$ h)	h
M	constante de desativação ($\geq \max_i dl_i - \min_i av_i$)	h

Tabela IV: Parâmetros derivados e de política, que não vêm de coluna do dataset.

C. Variáveis de decisão

- $x_{ij} \in \{0, 1\}$: 1 se o pacote i vai no caminhão j, 0 c.c.;
- $y_{jt} \in \{0, 1\}$: 1 se o caminhão j é do tipo t, 0 c.c.;
- $\tau_j \geq 0$ (h): instante de partida do caminhão j.

O modelo decide **o empacotamento** (quais pacotes viajam juntos), o **tipo** de cada caminhão e **quando** ele parte. Os slots são idênticos antes da escolha do tipo, o que cria simetria de permutação — quebrada por R9–R10.

D. Função objetivo

$$\min \sum_{j \in J} \sum_{t \in T} C_t \; y_{jt} \tag{FO}$$

Cada caminhão aberto custa o fixo da sua classe, $C_t = c_t \; d$; caminhões fechados não custam nada. Como C_t não depende da carga, o objetivo é minimizar o **custo de frota**, e a consolidação reduz o número de veículos. O custo **não** cresce com a ocupação: para o mesmo tipo, encher um caminhão é sempre melhor que abrir outro — é o custo fixo clássico do VSBPP [3], [4].

E. Restrições

1) Atribuição completa:

$$\sum_{j \in J} x_{ij} = 1 \quad \forall i \in I \tag{R1}$$

Cada pacote viaja em **exatamente um** caminhão; pacotes são indivisíveis.

2) Capacidade de área:

$$\sum_{i \in I} a_i \; x_{ij} \;\leq\; \sum_{t \in T} A_t \; y_{jt} \quad \forall j \in J \tag{R2}$$

3) Capacidade de peso:

$$\sum_{i \in I} w_i \; x_{ij} \;\leq\; \sum_{t \in T} W_t \; y_{jt} \quad \forall j \in J \tag{R3}$$

As duas capacidades (R2 e R3) valem **simultaneamente**: o caminhão só é viável se respeita área e peso. Na prática a área morde primeiro — a densidade mediana dos itens (797 kg/m²) supera a razão do caminhão de 16,5 m (671 kg/m²).

4) Um tipo por caminhão:

$$\sum_{t \in T} y_{jt} \leq 1 \quad \forall j \in J \tag{R4}$$

5) Vínculo pacote-caminhão:

$$x_{ij} \;\le\; \sum_{t\in T} y_{jt} \quad \forall i \in I,\ j \in J \tag{R5}$$

Caminhões fechados ficam vazios; por R4, o lado direito é binário.

6) Partida após a disponibilidade:

$$\tau_j \;\ge\; av_i - M\,(1 - x_{ij}) \quad \forall i \in I,\ j \in J \tag{R6}$$

7) Janela de consolidação:

$$\tau_j \;\le\; av_i + \Delta + M\,(1 - x_{ij}) \quad \forall i \in I,\ j \in J \tag{R7}$$

8) Chegada dentro do prazo:

$$\tau_j \;\le\; dl_i - t_{\text{viagem}} + M\,(1 - x_{ij}) \quad \forall i \in I,\ j \in J \tag{R8}$$

Juntas, R6–R8 caracterizam a viabilidade temporal do conjunto S_j de pacotes do caminhão j:

$$\max_{i\in S_j} av_i \;\le\; \tau_j \;\le\; \min\Big(\min_{i\in S_j} av_i + \Delta,\ \min_{i\in S_j} dl_i - t_{\text{viagem}}\Big),$$

ou seja, S_j é temporalmente viável se, e somente se,

$$\max_{i\in S_j} av_i - \min_{i\in S_j} av_i \le \Delta \quad \text{e} \quad \max_{i\in S_j} av_i + t_{\text{viagem}} \le \min_{i\in S_j} dl_i.$$

A escolha canônica é $\tau_j = \max_i av_i$: o caminhão parte assim que o último pacote fica pronto, o que maximiza a folga de prazo.

9) Quebra de simetria:

$$x_{ij} = 0 \quad \forall i \in I,\ j \in J:\ j > i \tag{R9}$$

R9 não exclui soluções: dada qualquer solução viável, ordene os slots abertos pelo menor índice de pacote que contêm e renumere-os $1, 2, \ldots$ nessa ordem. O k-ésimo slot aberto contém somente pacotes de índice $\ge k$, pois os $k-1$ slots anteriores já consomem $k-1$ pacotes de índice menor; logo o slot j só recebe pacotes $i \ge j$. A renumeração não altera custo nem viabilidade, de modo que toda solução tem uma representação que satisfaz R9 e o ótimo é preservado (a hipótese $m \le n$ garante que há rótulos suficientes). Restrições de ordenação desse tipo são a forma mais simples de quebra de simetria em formulações de empacotamento [16], [20].

A quebra propiciada por R9 é **parcial**: ela ordena os caminhões *abertos* pelo menor índice de pacote que transportam, mas a ordenação é uma desigualdade ($\min(S_j) \ge j$) e não uma igualdade: um caminhão cujo menor pacote tem índice maior que a posição que ocupa pode deslizar para uma posição posterior sem violar R9. Ela também não diz nada sobre a posição dos $m - o$ slots vazios, que permanecem permutáveis entre si. Cada plano de carregamento conserva, assim, várias representações de custo e de limitante idênticos — cópias que o *branch-and-bound* não consegue distinguir nem podar, pois o limitante de uma não serve para refutar a outra [16], [17].

10) Ordenação dos caminhões abertos:

$$\sum_{t\in T} y_{jt} \;\ge\; \sum_{t\in T} y_{j+1,t} \quad \forall j \in \{1, \ldots, m-1\} \tag{R10}$$

R10 exige que os caminhões abertos ocupem um prefixo de J; é a ordenação já empregada em formulações de empacotamento com caixas idênticas [20] e, como R9, não exclui soluções: dados os caminhões de uma solução viável, rotulando-os em ordem crescente do menor pacote que contêm, o k-ésimo recebe pacotes de índice $\ge k$ (satisfaz R9) e os o abertos passam a ser os slots $1, \ldots, o$ (satisfazem R10) — trocar um caminhão aberto por um slot vazio preserva viabilidade e custo. Combinadas, R9 e R10 fixam o **padrão de abertura** — os $m - o$ slots vazios são exatamente os últimos — mas não fixam a rotulagem dos caminhões abertos entre si. A simetria residual fica restrita aos caminhões *folgados*, cujo menor pacote tem índice maior que a posição que ocupam. Com $I = \{1, \ldots, 6\}$, $m = 6$ e o plano $\{1,2,3\}$, $\{4\}$, $\{5\}$, $\{6\}$, os três últimos ocupam os slots 2–4 em qualquer das 3! ordens, todas de custo idêntico. R10 não altera o ótimo — é a mesma frota, com o mesmo custo — mas reduz o esforço de prova, e é por isso que a incorporamos ao modelo.

11) Simetria entre pacotes idênticos: Com R9–R10 o conjunto de slots abertos é canônico. Além da ordem residual dos caminhões *folgados*, resta a simetria entre pacotes indistinguíveis — mesmos valores de área, peso, disponibilidade e prazo. Ela não é um detalhe nos dados reais: em `order_large.csv`, 4.631 dos 4.635 pacotes pertencem a rotas com repetição, e há rota com 276 pacotes e apenas 4 tipos distintos. A quebra adotada ordena cada grupo de idênticos pela posição que ocupam,

$$\sum_{j\in J} j\, x_{ij} \;\le\; \sum_{j\in J} j\, x_{i'j} \tag{R11}$$

para todo par consecutivo $i < i'$ de pacotes idênticos. Verificamos por enumeração exaustiva, em 70 instâncias minúsculas com repetições, que R11 não exclui solução ótima (desvio nulo contra o ótimo irrestrito), e medimos o ganho: em instâncias sintéticas com grupos de 4 a 5 cópias os nós caem de 38 para 9 e de 15.992 para 808, sem alterar o ótimo, e R11 é vacuamente inócua quando não há repetição (335 nós com e sem ela). Em subamostras reais de 30 a 40 pacotes o quadro se repete em parte — os nós caem de 65 para 11 (`City_37`) e de 231 para 19 (`City_58`) —, mas em outras não há mudança ou há piora (13 para 26 nós em `City_18`, $n = 40$), e o tempo **não** acompanha em todos os casos, porque as linhas extras pesam no pré-processamento de modelos pequenos. Esses custos não justificam abrir mão do corte: R11 **integra o modelo** e é aplicada em todas as instâncias, inclusive nas pequenas. Uma instância que resolve no nó raiz não tem tempo de prova a economizar, e nas maiores — onde o tempo de fato importa — o corte de nós é o que domina. Demonstração em `scripts/package_symmetry_benchmark.py`.

F. Sumário do modelo

De forma compacta, o modelo de otimização inteira mista é:

$$\min \sum_{j \in J} \sum_{t \in T} C_t \, y_{jt} \quad \text{s.a.} \quad \text{R1–R11}, \quad x_{ij},\ y_{jt} \in \{0,1\},\ \tau_j \geq 0,$$

com (I, T, J), (a_i, w_i), (A_t, W_t, C_t), (av_i, dl_i), t_{viagem} e Δ conforme declarado nas subseções anteriores.

IV. Declaração de uso de inteligência artificial

Este trabalho empregou ferramentas de inteligência artificial (IA) generativa como **apoio** a atividades de pesquisa, sempre sob supervisão humana e com responsabilidade integral do autor pelo conteúdo final. Esta seção declara quais ferramentas foram usadas, como foram configuradas e quais controles existem sobre o seu uso.

A. Escopo do uso

A IA foi utilizada em: (i) busca, triagem e conferência de literatura (identificação de trabalhos, verificação de DOI e de dados bibliográficos); (ii) apoio à implementação e à depuração do código de modelagem e de solução (PuLP/HiGHS) e dos scripts de tratamento de instâncias; (iii) revisão de redação, consistência terminológica e conversão de unidades; e (iv) checagem de consistência interna entre texto, tabelas e resultados reportados.

Não foram produzidos por IA os dados, a instância, os resultados numéricos nem as conclusões: toda solução reportada provém de execuções dos solvers sobre os arquivos em `data/raw/`, reproduzíveis a partir do código versionado, e cada número citado no texto foi reconferido contra a saída correspondente. A formulação do modelo, a escolha das instâncias e a interpretação dos resultados são de responsabilidade humana, assim como a decisão final sobre o que consta no relatório.

B. Ambiente e configuração

Todo o uso ocorreu por meio do **OpenCode**, um *agent harness* de código aberto executado localmente, cuja configuração é versionada de forma declarativa (Nix + Home Manager) no repositório público `o-abreu/nix-config`, no estado do commit `90b48b2` (28 out. 2026).[1] A configuração do agente é, portanto, auditável: os caminhos abaixo são relativos a `home/features/programs/cli/opencode/`.

- `default.nix` — habilitação do OpenCode e preferências da interface;
- `agents.nix` — instruções globais do agente (`AGENTS.md`), incluindo a regra de **nunca** instalar pacotes de forma imperativa, restringindo o ambiente ao que está declarado no flake;
- `agents/primary/research.nix` — definição do agente `research`, o único autorizado a acionar o ferramental de pesquisa bibliográfica;
- `tools/asta/default.nix` e `tools/asta/_skills.nix` — *skills* e CLI do **Asta** (AllenAI), *vendorizados* do projeto `allenai/asta-plugins`;
- `tools/zotero.nix` — acesso ao **Zotero** local via `zotero-mcp`/`zotero-cli` sobre a API HTTP local do Zotero;
- `mcp/` — definição dos servidores MCP habilitados;
- `providers/` — provedores de modelos de linguagem.

As tarefas de pesquisa usam o agente `research`, que tem permissão explícita apenas para as ferramentas de literatura; as mesmas ferramentas (Asta, Zotero e Kaggle) são negadas globalmente para os demais agentes. Essa separação é deliberada: ela concentra em um único perfil, com escopo restrito, o acesso a bases bibliográficas e a execução de comandos de pesquisa.

C. Recuperação de literatura

A busca bibliográfica usou as *skills* Asta `find-literature`, `semantic-scholar`, `pdf-download` e `literature-report`, apoiadas na API do Semantic Scholar e em PDFs de acesso aberto, além da *skill* `check-claims` para revisar a aderência entre afirmações e evidências. A bibliografia foi mantida na coleção local do Zotero *Programação Matemática — Bin Packing (VSBPP)*, com subcoleção *Datasets* para os dados e *softwares*. Toda entrada do `refs.bib` foi verificada quanto a autoria, veículo, ano e DOI; referências sugeridas por IA e não confirmadas em fonte primária foram descartadas.

D. Servidores MCP e modelos

Os servidores MCP habilitados no ambiente são `context7` (documentação de bibliotecas), `kaggle` (acesso ao conjunto de dados), `github`, `nixos`, `devenv` e `morph-mcp`. Os modelos de linguagem utilizados foram o **DeepSeek V4.1 Flash** e o **GLM 5.3 Flash**, acessados por API por meio dos provedores declarados em `providers/` (`deepseek` e `opencode-go`); a escolha entre os dois modelos é feita por sessão.

E. Controles e verificação

Três controles foram adotados: (i) ambiente reprodutível — dependências declaradas (`devenv.nix`, `requirements.txt`) e nenhuma instalação imperativa, de modo que todo código gerado com apoio de IA é executado no mesmo ambiente descrito no repositório; (ii) verificação de fontes — nenhuma referência bibliográfica entra no texto sem conferência em fonte primária; (iii) reconferência de resultados — os números do relatório são lidos das saídas do solver, não transcritos de respostas do modelo.

O autor revisou integralmente o texto, o código e os resultados aqui apresentados. Eventuais erros remanescentes são de sua responsabilidade.

[1] Repositório público: https://github.com/o-abreu/nix-config. Configuração do agente em `home/features/programs/cli/opencode/`; regras específicas deste projeto em `AGENTS.md`, na raiz do repositório do trabalho.

V. Referências

[1] R. M. Karp, "Reducibility among Combinatorial Problems", Princeton University, Complexity of Computer Computations, Plenum Press, 1972.

[2] M. R. Garey e D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.

[3] D. K. Friesen e M. A. Langston, "Variable Sized Bin Packing", *SIAM Journal on Computing*, vol. 15, nº 1, p. 222–230, 1986.

[4] J. Kang e S. Park, "Algorithms for the Variable Sized Bin Packing Problem", *European Journal of Operational Research*, vol. 147, nº 2, p. 365–372, 2003.

[5] Brunel University London, "Supply Chain Logistics Problem Dataset". Kaggle, https://www.kaggle.com/datasets/omarstat2/supply-chain-logistics-data, originally https://brunel.figshare.com/articles/dataset/Supply_Chain_Logistics_Problem_Dataset/7558679, 2026.

[6] Microsoft, "Large-Scale Route Optimization Accelerator". GitHub, https://github.com/microsoft/dstoolkit-route-optimization, 2022.

[7] mexwell, "Large-Scale Route Optimization Dataset". Kaggle, https://www.kaggle.com/datasets/mexwell/large-scale-route-optimization, 2026.

[8] F. D. Murgolo, "An Efficient Approximation Scheme for Variable-Sized Bin Packing", *SIAM Journal on Computing*, vol. 16, nº 1, p. 149–161, 1987.

[9] L. Epstein e A. Levin, "An APTAS for Generalized Cost Variable-Sized Bin Packing", *SIAM Journal on Computing*, vol. 38, nº 1, p. 411–428, 2008.

[10] C. Chekuri e S. Khanna, "On Multidimensional Packing Problems", *SIAM Journal on Computing*, vol. 33, nº 4, p. 837–851, 2004.

[11] Q. Hu, L. Wei, e A. Lim, "The two-dimensional vector packing problem with general costs", *Omega*, vol. 74, p. 59–69, 2018.

[12] G. J. Woeginger, "There is no asymptotic PTAS for two-dimensional vector packing", *Information Processing Letters*, vol. 64, nº 6, p. 293–297, 1997.

[13] W. Fernandez de la Vega e G. S. Lueker, "Bin packing can be solved within $1 + \varepsilon$ in linear time", *Combinatorica*, vol. 1, nº 4, p. 349–355, 1981.

[14] N. Karmarkar e R. M. Karp, "An efficient approximation scheme for the one-dimensional bin-packing problem", em *23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)*, 1982, p. 312–320.

[15] E. G. Coffman Jr., M. R. Garey, e D. S. Johnson, "Approximation Algorithms for Bin Packing: A Survey", em *Approximation Algorithms for NP-Hard Problems*, D. S. Hochbaum, Org. Boston: PWS Publishing, 1997.

[16] F. Margot, "Symmetry in Integer Linear Programming", em *50 Years of Integer Programming 1958–2008*, M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, e L. A. Wolsey, Orgs. Berlin: Springer, 2010, p. 647–686.

[17] J. Ostrowski, J. Linderoth, F. Rossi, e S. Smriglio, "Orbital Branching", *Mathematical Programming*, vol. 126, nº 1, p. 147–178, 2011.

[18] H. D. Sherali e J. C. Smith, "Improving Discrete Model Representations via Symmetry Considerations", *Management Science*, vol. 47, nº 10, p. 1396–1407, 2001.

[19] V. Kaibel e M. E. Pfetsch, "Packing and Partitioning Orbitopes", *Mathematical Programming*, vol. 114, nº 1, p. 1–36, 2008.

[20] K. H. Salem e Y. Kieffer, "An Experimental Study on Symmetry Breaking Constraints Impact for the One Dimensional Bin-Packing Problem", em *Proceedings of the 2020 Federated Conference on Computer Science and Information Systems (FedCSIS)*, 2020, vol. 21, p. 317–326.